



Inspiring Excellence

Course Title: Programming Language II

Course Code: CSE 111

Semester: Summer 2020

**Topic: Object-Oriented Programming (Encapsulation & Operator
Overloading)**

Table of Contents

1. Python - public, private:	1
2. Encapsulation:	3

1. Python - public, private:

Object-oriented languages control the access to class resources by public, private and protected keywords.

In python, it doesn't have any mechanism that effectively restricts access to any instance variable or method. Python prescribes a convention of prefixing the name of the variable/method with single or double underscore to emulate the behavior of protected and private access specifiers.

Before starting python encapsulation, let's discuss how python handle public, private members-

1.1. Public member:

Public members (generally methods declared in a class) are accessible from outside the class. The object of the same class is required to invoke a public method. All members in a Python class are public by default. Any member can be accessed from outside the class environment.

Here, in the following example, variables can call outside of call and also can change the variables.

```
class Student:
    def __init__(self, name, id):
        self.name=name
        self.id=id
s1=Student("Kiran",18101)
print(s1.id)
s1.id=18202
print(s1.id)
```

Output:

18101

1820

1.2. Private member:

Private members of a class are denied access from the environment outside the class. They can be handled only from within the class.

Private members are similar to protected members, the difference is that the class members declared private should neither be accessed outside the class nor by any base class. In Python, there is no existence of Private instance variables that cannot be accessed except inside a class. However, to define a private member prefix the member name with double underscore “__” (double underscore).

Here in following example, attribute cannot be change outside of the class as it is assigned as private attribute.

```
class Student:

    def __init__(self):
        self.__id = 18101 # private attribute
        self.__name = "Kiran" # private attribute

    def info(self):
        print(self.__id)

s1 = Student()
s1.info()
s1.__id = 18202
# will not change variable because its private
s1.info()
```

Output:

18101

18101

Also from the following example we can differentiate public, private attributes. Here public attribute can access outside the class but in case of private attribute it shows an error.

```
class Robot(object):  
    def __init__(self):  
        self.a = 123 #public attribute  
        self.__c = 123 #private attribute  
  
obj = Robot()  
print(obj.a)  
print(obj.__c)
```

Output:

```
123
```

```
Traceback (most recent call last):
```

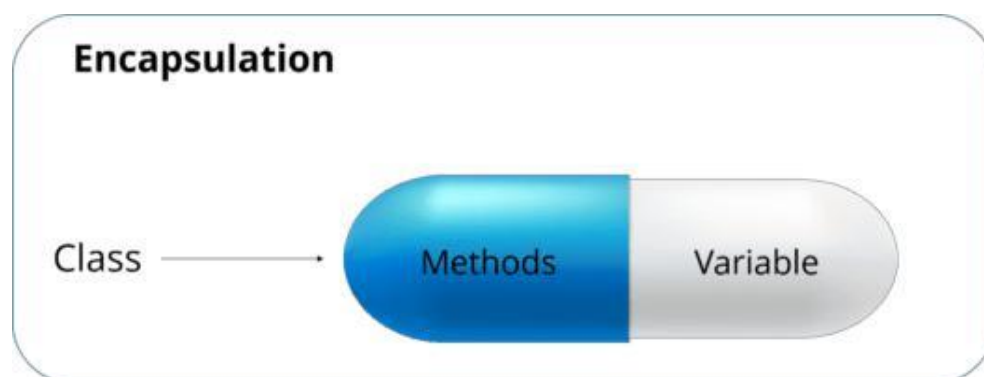
```
  File "test.py", line 10, in  
    <module>: print(obj.__c)
```

```
AttributeError: 'Robot' object has no attribute '__c'
```

2. Encapsulation:

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of wrapping data and the methods that work on data within one unit. This puts restrictions on accessing variables and methods directly and can prevent the accidental modification of data. To prevent accidental change, an object's variable can only be changed by an object's method. Those type of variables are known as **private variable**.

A class is an example of encapsulation as it encapsulates all the data that is member functions, variables, etc.



Example:

Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales. Now there may arise a situation when for some reason an official from the finance section needs all the data about sales in a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact some other officer in the sales section and then request him to give the particular data. This is what encapsulation is. Here the data of the sales section and the employees that can manipulate them are wrapped under a single name “sales section”. As using encapsulation also hides the data. In this example, the data of any of the sections like sales, finance or accounts are hidden from any other section. So encapsulation provides security by hiding the data from the outside world.

In Python, Encapsulation can be achieved by declaring the data members of a class either as **private** or **protected**.

Let us see how access modifiers help in achieving Encapsulation.

2.1 Encapsulation Using Private Members:

If we declare any variable or method as private, then they can be accessed only within the class in which they are defined.

In the below example, 'length' and 'breadth' are the two private variables declared and can be accessed within the class 'Rectangle'.

```
class Rectangle:
    __length = 0 #private variable
    __breadth = 0#private variable
    def __init__(self): #constructor
        self.__length = 5
        self.__breadth = 3
    #printing values of the private variable within the
        class print(self.__length)
        print(self.__breadth)

rec = Rectangle() #object created for the class 'Rectangle'
#printing values of the private variable outside the class using
the object created for the class 'Rectangle'
print(rec.length)
print(rec.breadth)
```

Output:

5

3

Traceback (most recent call last):

File "main.py", line 11, in

<module> print(rec.__length)

AttributeError: 'Rectangle' object has no attribute '__length'

Since we have accessed private variables in the main() method, i.e., outside the class 'Rectangle', we got an error. Hence in the above program, Encapsulation is achieved using the private variables 'length' and 'breadth'.